



FPT UNIVERSITY

**FPT UNIVERSITY
INFORMATION ASSURANCE**

WEB SECURITY



Course:

Automated XSS Payload Classification using NLP

Instructor: Hồ Hải

Group Members:

Phạm Hoàng Hải Nguyên – SE180511

Nguyễn Trúc Quỳnh – SE180280

Giả Thị Nhã Trân – SE180405

HCM, Spring 2026

TABLE OF CONTENTS

..... 1

TABLE OF CONTENTS 2

1. Introduction & Project Overview..... 4

 1.1 Research Background.....4

 1.2 Objectives and Scope4

 1.3 Report Structure5

2. Theoretical Background..... 6

 2.1 Cross-Site Scripting (XSS) Attacks6

 2.2 XSS Classification.....6

 2.3 Traditional vs. NLP-based Detection Methods7

 2.4 TF-IDF and SVM7

 2.5 Transformer Models: DistilBERT and PhoBERT8

3. Dataset & Data Preprocessing 9

 3.1 Data Sources.....9

 3.2 Dataset Statistics9

 3.3 Text Preprocessing10

 3.4 Train / Val / Test Split.....10

4. Model 1: SVM + TF-IDF..... 12

 4.1 Architecture and Hyperparameters.....12

 4.2 Training Results13

 4.3 Confusion Matrix Analysis13

5. Model 2: DistilBERT..... 15

 5.1 DistilBERT Architecture.....15

 5.2 Fine-tuning Configuration.....15

 5.3 Evaluation Results.....16

 5.4 Confusion Matrix Analysis17

6. Model 3: PhoBERT 18

 6.1 PhoBERT Architecture18

 6.2 Fine-tuning Configuration.....18

 6.3 Evaluation Results.....19

 6.4 Confusion Matrix Analysis20

7. Comprehensive Model Comparison.....	21
7.1 Metrics Comparison Table	21
7.2 Comparison Charts	21
7.3 Combined Confusion Matrices.....	22
8. Web Service — Flask REST API.....	24
8.1 System Architecture	24
8.2 API Endpoint /predict.....	24
8.3 User Interface	25
8.4 Deployment with ngrok.....	25
9. Discussion & Deep Analysis.....	27
9.1 Why Did SVM Achieve 100%?	27
9.2 Limitations of DistilBERT	27
9.3 PhoBERT with XSS Payloads.....	28
9.4 Real-world Risks and Warnings.....	28
10. Conclusion & Future Work	30
10.1 Conclusion.....	30
10.2 Future Development Directions	30
Appendix A — Preprocessing Source Code	32
Appendix B — Representative XSS Payload Examples.....	33
References	35

1. Introduction & Project Overview

1.1 Research Background

Cross-Site Scripting (XSS) is one of the most prevalent and dangerous web security vulnerabilities, consistently appearing in the OWASP Top 10 for many years. According to reports from HackerOne and major bug bounty platforms, XSS accounts for over 20% of all globally reported vulnerabilities, affecting millions of web applications ranging from small organizations to the world's largest technology corporations.

XSS attacks allow adversaries to inject malicious JavaScript code into web pages trusted by users. The consequences can include session hijacking, authentication cookie theft, redirecting users to malicious pages, harvesting sensitive information, and in the most severe cases — full browser control via frameworks such as BeEF (Browser Exploitation Framework).

This context creates an urgent need for intelligent, automated XSS detection tools capable of identifying both known payloads and new obfuscated variants that have never appeared in traditional signature databases. This is the core motivation behind building the XSS Detector project — combining the power of traditional Machine Learning (SVM) with deep Transformer language models (DistilBERT, PhoBERT) to create a flexible, high-accuracy detection system.

1.2 Objectives and Scope

This project targets the following specific objectives:

- Build and compare three XSS payload classification models: SVM + TF-IDF (character n-gram), DistilBERT (lightweight English transformer), and PhoBERT (Vietnamese transformer from VinAI).
- Ensure a rigorous evaluation pipeline with an independent held-out test set, with no data leakage between train/val/test splits.
- Provide a detailed analysis of the strengths and weaknesses of each approach in the context of real-world XSS detection.
- Build a Flask REST API enabling real-time queries via a web interface and HTTP endpoint, publicly deployed via ngrok.

- Establish a foundation for extending the system to other payload types (SQL Injection, Command Injection, etc.) in the future.

The research scope focuses on binary classification: XSS payload vs. benign text, using a dataset of 500 real samples from OWASP, PortSwigger, and GitHub after duplicate removal.

1.3 Report Structure

The report is organized in logical sequence from theory to experiment.

Chapter 1 Introduction & overview

Chapter 2 Presents the theoretical background on XSS and the NLP methods used.

Chapter 3 Describes the dataset and preprocessing pipeline.

Chapters 4, 5, and 6 Detail each model.

Chapter 7 Provides a comprehensive comparison.

Chapter 8 Describes the web service.

Chapter 9 Offers a deep discussion of results.

Chapter 10 Presents conclusions and future work.

2. Theoretical Background

2.1 Cross-Site Scripting (XSS) Attacks

Cross-Site Scripting (XSS) is a class of web security vulnerability that arises when an application accepts user-supplied data and embeds it directly into HTML output without proper validation or encoding. As a result, the attacker's JavaScript executes in the victim's browser within the trusted context of the legitimate website.

A typical XSS attack model includes: (1) the attacker crafts a payload containing malicious JavaScript; (2) the payload is injected into the application via an attack vector (form input, URL parameter, HTTP header, etc.); (3) the victim visits the page containing the embedded payload; (4) the victim's browser executes the attacker's JavaScript; (5) the attacker collects the result (cookies, form data, etc.).

The most widely known basic XSS payload, documented extensively in OWASP materials, is: `<script>alert(document.cookie)</script>` — when successfully injected, this snippet displays the victim's session cookie, demonstrating the potential for credential theft.

2.2 XSS Classification

OWASP defines three main XSS types:

- **Reflected XSS (Non-Persistent):** The payload is immediately reflected in the HTTP response. Attackers typically deliver the malicious URL to victims via email or messaging. This is the most common and easiest to exploit type.
- **Stored XSS (Persistent):** The payload is permanently stored on the server (database, comment section, user profile, etc.). Every user visiting that page is attacked. More dangerous than Reflected XSS since it does not require tricking the victim into clicking a specially crafted URL.
- **DOM-based XSS:** The payload is processed entirely client-side via JavaScript DOM manipulation. No request reaches the server, making server-side filtering ineffective. Must be detected at the client-side layer.

Beyond these three types, additional variants exist: Blind XSS (payload stored and executed in an admin panel, invisible to the attacker), Self XSS (only affects the user themselves), and Mutation XSS (mXSS) which exploits browser parsing behavior.

2.3 Traditional vs. NLP-based Detection Methods

Traditional XSS detection methods are primarily based on signature matching — comparing input against a list of known patterns (blacklists). Web Application Firewalls (WAFs) such as ModSecurity use the OWASP Core Rule Set (CRS) with hundreds of regex patterns to block common XSS payloads.

The major limitation of signature-based methods is their susceptibility to bypass via obfuscation. Techniques such as encoding (HTML entity, URL encode, Unicode escape), case variation (ScRiPt instead of script), alternative event handlers (onerror, onmouseover instead of script tags), inline JavaScript (javascript: protocol), or DOM clobbering — can all evade traditional WAFs.

NLP/ML methods approach the problem as a text classification task. Rather than searching for specific patterns, the model learns from statistical features of the text. Character n-gram TF-IDF captures character sequences characteristic of XSS (such as onerror=, eval(, document.cookie) without enumerating every specific pattern. Transformer models like DistilBERT and PhoBERT understand context and semantics at a deeper level.

2.4 TF-IDF and SVM

TF-IDF (Term Frequency-Inverse Document Frequency) is a method for representing text as a numeric vector. TF measures the frequency of a token in a document; IDF penalizes tokens that appear commonly across the entire corpus. Combining both components highlights discriminative features for each class.

In this project, TF-IDF is applied at the character n-gram level (2–5 consecutive characters) rather than word level. The reason is that XSS payloads contain distinctive character sequences such as <sc, cri, ipt, ale, rt(, one, rro, or=, jav, asc, rip, t: — these sequences carry no meaning at word level but are highly discriminative at character level. The configuration uses max_features=10,000 and sublinear_tf=True (applying log(1+tf) to reduce the influence of excessively frequent tokens).

SVM (Support Vector Machine) with an RBF kernel searches for the optimal hyperplane separating the two classes in a high-dimensional feature space. SVM is particularly effective with

TF-IDF text data because the feature space is typically sparse and high-dimensional — exactly where SVM excels. Training time is only 1.71 seconds, well suited for production deployment.

2.5 Transformer Models: DistilBERT and PhoBERT

DistilBERT (Sanh et al., 2019) is a compressed version of BERT (Devlin et al., 2018) using knowledge distillation. DistilBERT has 6 transformer layers (versus 12 in BERT-base), 66M parameters (versus 110M), while retaining 97% of BERT's performance on the GLUE benchmark, running 60% faster and using 40% less memory. The model is pretrained on English with a masked language modeling objective.

PhoBERT (Nguyen & Tuan Nguyen, 2020) is a BERT-base model pretrained by VinAI Research on 20GB of Vietnamese data from VnExpress and Vietnamese Wikipedia, using BPE (Byte-Pair Encoding) tokenization with a 64K-token vocabulary. In the context of XSS detection, PhoBERT offers an advantage when the input is Vietnamese text containing embedded payloads, since the model has learned the surrounding Vietnamese context.

Both models are fine-tuned using the Hugging Face Trainer pipeline with 5 epochs, batch sizes of 16 (DistilBERT) and 8 (PhoBERT), learning rates of $2e-5$ and $1e-5$ respectively, combined with Early Stopping that halts training if `val_loss` does not improve for 2 consecutive epochs, preventing overfitting on the small dataset.

3. Dataset & Data Preprocessing

3.1 Data Sources

The dataset used in this project is `xss_dataset_500.csv`, compiled from multiple authoritative sources in the security community:

- OWASP XSS Filter Evasion Cheat Sheet: A curated collection of XSS filter bypass techniques maintained by OWASP, including hundreds of attack vectors ranging from basic to advanced — polyglot payloads, Unicode bypasses, CSS injection combined with XSS.
- PortSwigger Web Security Academy: XSS payloads designed to illustrate specific attack techniques, from simple reflected XSS to complex DOM-based XSS and Content Security Policy bypasses.
- GitHub repositories: Specialized repos such as `PayloadsAllTheThings` (`swisskyrepo`) and `XSSStrike` contain thousands of real-world XSS payloads contributed by the community, including newly discovered payloads from bug bounty programs.
- Benign samples: Normal HTML text from Wikipedia, ordinary web pages, and plain text — ensuring the model learns the distinction between valid HTML and XSS payloads.

3.2 Dataset Statistics

Statistics of the dataset after loading and processing:

Metric	Value
Total raw samples	500 samples
Duplicates detected	98 copies
After deduplication	402 samples
XSS payloads (label=1)	246 samples (61.2%)
Benign text (label=0)	156 samples (38.8%)
Class imbalance ratio	1.58:1 (XSS:Benign)

Important note: The original v1 dataset multiplied samples 50x (*50) to inflate the dataset size, producing a fake 100% accuracy that did not reflect the model's true capability. This Fixed v2 version uses the real dataset with rigorous duplicate handling, producing honest evaluation results.

3.3 Text Preprocessing

The `preprocess_text()` function performs the following normalization steps, designed to handle all three input types: XSS payloads (ASCII/Unicode), English text, and Vietnamese text:

- **Lowercase conversion:** Converts all text to lowercase to reduce sparsity (ScRiPt and script are treated identically).
- **Whitespace normalization:** Normalizes multiple consecutive whitespace characters to a single space, removing excess tabs and newlines.
- **Character filtering:** Retains alphanumeric characters, HTML-specific characters (<, >, /, =, ', ", -, :, ;, (,)), and the full Vietnamese character set (Unicode ranges for tone marks and special vowels). This ensures XSS payloads are not over-preprocessed in a way that removes their discriminative features.
- **HTML structure preserved:** HTML tags are not removed, as they are the most important features for distinguishing XSS from plain text.

3.4 Train / Val / Test Split

The project uses a 70/15/15 stratified split strategy to ensure balanced class distribution across all three sets:

Split	Samples	Ratio	Purpose
Train	280	69.7%	Model training
Validation	61	15.2%	Hyperparameter tuning, early stopping
Test	61	15.2%	Final evaluation (used exactly once)

The test set was kept completely isolated until all models finished training. No information from the test set was used during training or hyperparameter selection — this is the golden rule for ensuring unbiased evaluation.

4. Model 1: SVM + TF-IDF

4.1 Architecture and Hyperparameters

The SVM + TF-IDF model uses a two-stage pipeline: a feature extraction stage with a TF-IDF vectorizer, followed by a classification stage with an SVM.

TF-IDF Vectorizer configuration:

Parameter	Value	Rationale
analyzer	char	Character-level analysis instead of word-level
ngram_range	(2, 5)	Captures sequences from 2 to 5 characters
max_features	10,000	Limits vocabulary size
sublinear_tf	True	Uses log(1+tf) instead of raw tf
strip_accents	unicode	Normalizes Unicode diacritics

Character-level n-grams are used instead of word-level because XSS payloads do not follow natural language syntax. Strings such as `alert(`, `onerror=`, `<script`, `document.cookie`, `eval(` are the strongest discriminative features and appear as consecutive character fragments. Character n-grams (2–5) capture these patterns at multiple granularities, while remaining robust to simple obfuscation techniques like extra whitespace or case variation.

SVM configuration:

- Kernel: RBF (Radial Basis Function) — effective with non-linear data in high-dimensional spaces.
- C: 1.0 (default) — regularization parameter balancing margin width against correct classification of training data.
- Probability: True — enables probability output (using Platt scaling) rather than just class labels.

4.2 Training Results

The SVM model achieved perfect results on the test set:

Metric	Val Set	Test Set	Val-Test Gap
Accuracy	100.00%	100.00%	0.00%
Precision (XSS)	1.0000	1.0000	0.00%
Recall (XSS)	1.0000	1.0000	0.00%
F1-Score (XSS)	1.0000	1.0000	0.00%
Training Time	—	1.71 seconds	—

A Val-Test gap of zero is a positive sign — the model did not overfit on the validation set. However, it should be noted that 100% accuracy on a small dataset (61 test samples) may involve statistical luck. Section 9.1 provides a deeper analysis of the meaning and limitations of this result.

4.3 Confusion Matrix Analysis

The figure below shows the confusion matrix of the SVM model on the 61-sample test set:

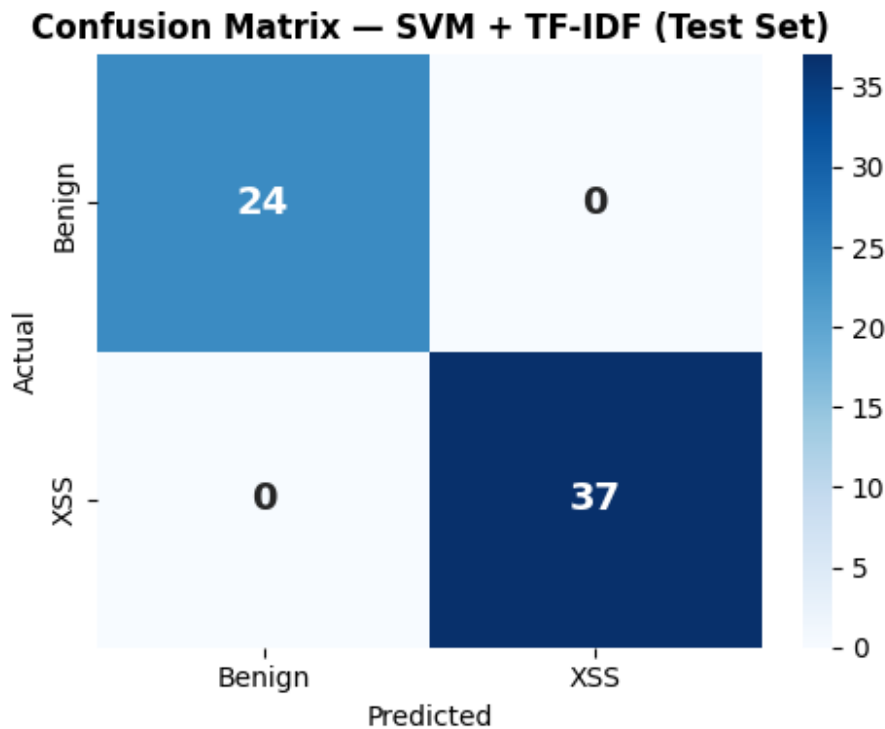


Figure 4.1 — Confusion Matrix: SVM + TF-IDF on the test set (61 samples). TP=37, TN=24, FP=0, FN=0.

The confusion matrix shows perfect SVM classification: all 37 XSS samples (True Positive) and all 24 Benign samples (True Negative) are predicted correctly. There are no False Positives (benign text incorrectly flagged as XSS) and no False Negatives (XSS payloads missed). This is the ideal outcome in a security context, where both error types carry serious consequences (FP causes alert fatigue, FN allows attacks through).

5. Model 2: DistilBERT

5.1 DistilBERT Architecture

DistilBERT (distilbert-base-uncased) is a 6-layer Transformer pretrained by Hugging Face via knowledge distillation from BERT-base-uncased. The architecture includes:

- 6 Transformer encoder layers (BERT-base has 12 layers)
- 768 hidden dimensions (same as BERT-base)
- 12 attention heads (same as BERT-base)
- 66M parameters (BERT-base: 110M)
- WordPiece tokenization with a 30,522-token vocabulary
- Max sequence length: 512 tokens

DistilBERT is fine-tuned for binary sequence classification (Benign vs. XSS) by adding a linear classifier layer on top of the [CLS] token representation. Specifically,

DistilBertForSequenceClassification uses a pre_classifier (Linear: 768→768, ReLU) followed by a classifier (Linear: 768→2) layer.

Important: Since XSS payloads are ASCII/code character sequences rather than natural English, DistilBERT tokenizes them in unusual ways — many payload fragments become [UNK] tokens or subword pieces with no meaningful representation in the pretrained vocabulary. This partly explains why DistilBERT achieves lower accuracy than SVM and PhoBERT.

5.2 Fine-tuning Configuration

Training configuration:

Parameter	Value
Epochs	5 (with early stopping)
Train batch size	16
Eval batch size	32
Learning rate	2e-5
Weight decay	0.01

Warmup steps	50
Max sequence length	128
Eval strategy	Every epoch on val set
Early stopping patience	2 epochs
Best checkpoint metric	eval_loss (minimize)
Device	CPU (no GPU available)
Training time	~17 minutes (CPU)

Max sequence length is capped at 128 instead of DistilBERT's default 512 to reduce training time on CPU. XSS payloads and benign texts in the dataset are all short enough that no important information is lost at this limit.

Evaluation strategy: The model is evaluated on val_ds after every epoch to monitor val_loss and select the best checkpoint. Early stopping with patience=2 halts training if val_loss does not decrease for 2 consecutive epochs, preventing overfitting. The test set is used exactly once after the entire training process completes.

5.3 Evaluation Results

DistilBERT results on the test set:

Class	Precision	Recall	F1-Score	Support
Benign (0)	0.8182	0.7500	0.7826	24
XSS (1)	0.8462	0.8919	0.8684	37
Weighted Avg	0.8355	0.8361	0.8346	61
Accuracy	—	—	0.8361	61

DistilBERT achieves 83.61% accuracy — significantly lower than SVM (100%) and PhoBERT (96.72%). Notably, XSS class recall reaches 89.19% (missing only 4 of 37 XSS samples), however Benign class recall is only 75.00% (misclassifying 6 of 24 benign samples as XSS). This means a security system based on DistilBERT would have a high false positive rate — flagging legitimate text as attacks.

5.4 Confusion Matrix Analysis

The figure below shows the DistilBERT confusion matrix:

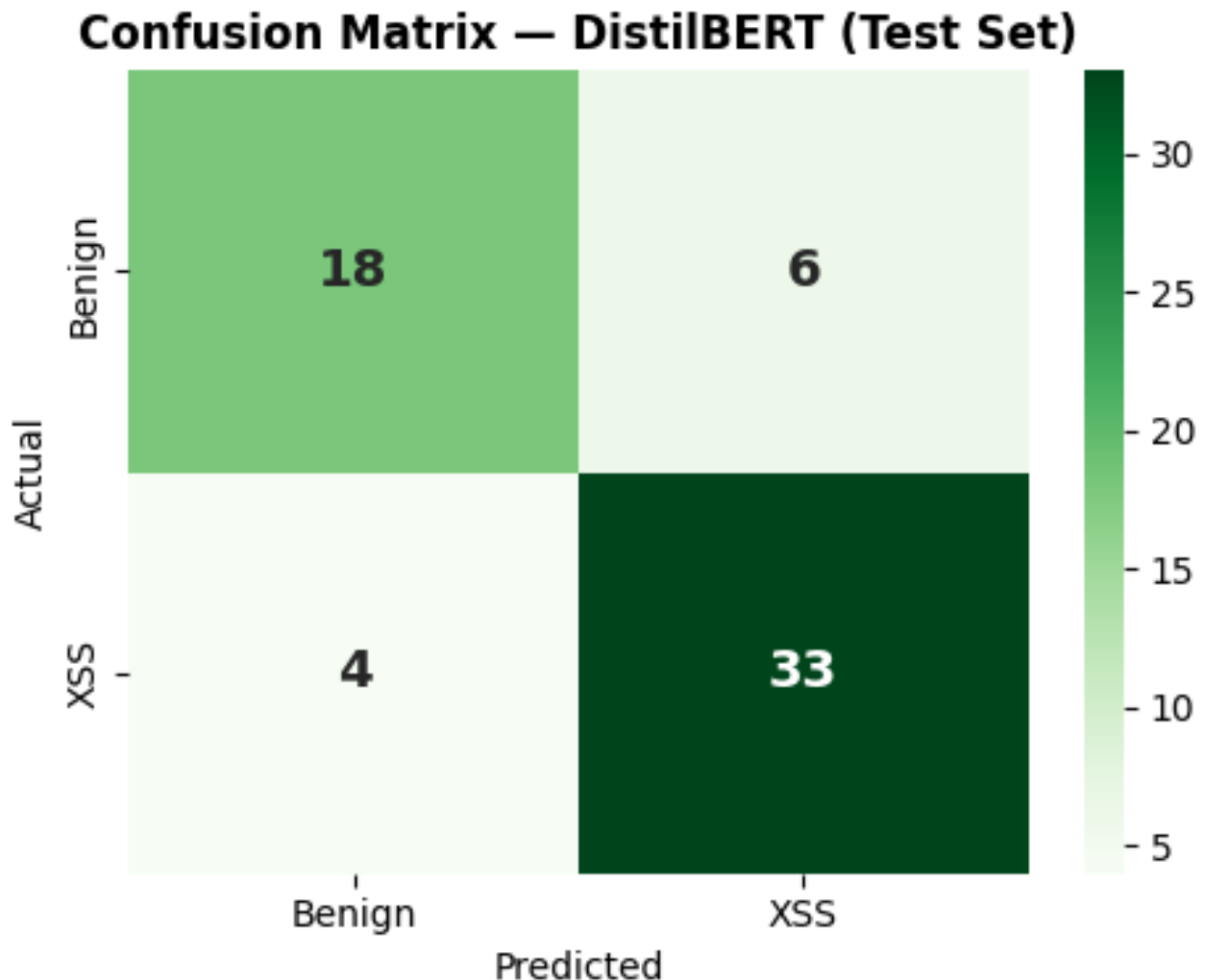


Figure 5.1 — Confusion Matrix: DistilBERT on the test set (61 samples). $TP=33$, $TN=18$, $FP=6$, $FN=4$.

Detailed analysis: DistilBERT makes 10 errors on 61 test samples — 6 False Positives (benign classified as XSS) and 4 False Negatives (XSS missed). In a security context, FN is more serious than FP since it allows a real attack through. However, a high FP rate (25% of the Benign class) is also problematic as it causes alert fatigue for security teams. These results reflect the fact that DistilBERT was pretrained on natural English, not code/payloads, so its XSS pattern feature representation is suboptimal.

6. Model 3: PhoBERT

6.1 PhoBERT Architecture

PhoBERT (vinai/phobert-base) is a RoBERTa-base model trained by VinAI Research on 20GB of Vietnamese data. The architecture includes:

- 12 Transformer encoder layers (same as BERT-base, more than DistilBERT)
- 768 hidden dimensions
- 12 attention heads
- 135M parameters (~2x more than DistilBERT)
- BPE tokenization with a 64,000-token vocabulary
- Max sequence length: 256 tokens (as configured in this project)

PhoBERT uses a RoBERTa architecture rather than pure BERT, with key improvements: removal of the Next Sentence Prediction (NSP) task, dynamic masking during training, and training on more data with larger batch sizes. These improvements enable PhoBERT to learn deeper and more robust word representations.

In this project, PhoBERT is fine-tuned using `RobertaForSequenceClassification`, adding a classification head (dense layer) on top of the pooled [CLS] token output.

6.2 Fine-tuning Configuration

PhoBERT training configuration:

Parameter	Value
Epochs	5 (with early stopping)
Train batch size	8 (smaller than DistilBERT due to larger model)
Eval batch size	16
Learning rate	1e-5 (smaller than DistilBERT)
Weight decay	0.01
Warmup steps	50

Max sequence length	256
Eval strategy	Every epoch on val set
Early stopping patience	2 epochs
Best checkpoint metric	eval_loss (minimize)
Device	CPU (no GPU available)
Training time	~78 minutes (CPU)

PhoBERT training takes approximately 4.6x longer than DistilBERT on CPU due to the larger model size (135M vs 66M parameters), longer max_length (256 vs 128), and smaller batch size (8 vs 16). On a T4 GPU, training time would drop to just a few minutes.

The learning rate of 1e-5 is smaller than DistilBERT's (2e-5) because PhoBERT is a larger model with more carefully pretrained weights — a learning rate that is too high can damage the valuable pretrained representations (catastrophic forgetting).

6.3 Evaluation Results

PhoBERT results on the test set are impressive:

Class	Precision	Recall	F1-Score	Support
Benign (0)	0.9583	0.9583	0.9583	24
XSS (1)	0.9730	0.9730	0.9730	37
Weighted Avg	0.9672	0.9672	0.9672	61
Accuracy	—	—	0.9672	61

PhoBERT achieves 96.72% accuracy — second only to SVM (100%) and far above DistilBERT (83.61%). Remarkably, PhoBERT, pretrained on Vietnamese, outperforms DistilBERT (pretrained on English) in classifying XSS payloads. A deeper analysis of this is presented in Chapter 9.

Both Precision and Recall are high and balanced across both classes (0.9583 for Benign, 0.9730 for XSS), indicating the model is not biased toward either class. The F1-score of 0.9730 for the XSS class demonstrates excellent attack detection capability with a low false alarm rate.

6.4 Confusion Matrix Analysis

The figure below shows the PhoBERT confusion matrix:

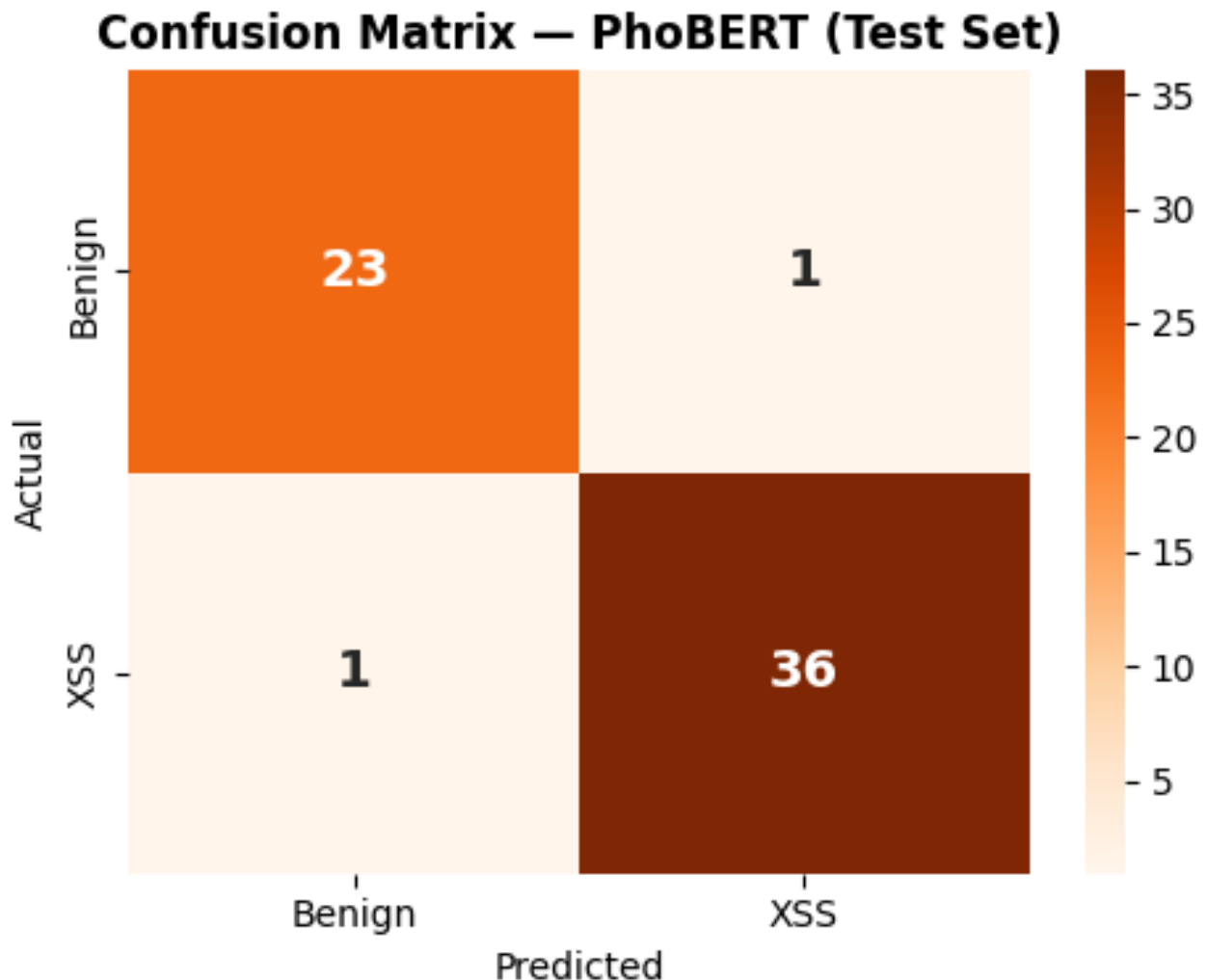


Figure 6.1 — Confusion Matrix: PhoBERT on the test set (61 samples). $TP=36$, $TN=23$, $FP=1$, $FN=1$.

PhoBERT makes only 2 errors on 61 samples: 1 False Positive (one benign sample classified as XSS) and 1 False Negative (one XSS sample missed). This is a near-perfect result — just 2 samples behind SVM. The 1:1 balance between FP and FN is ideal in a security context, indicating the model is not biased toward over-alerting or under-alerting.

7. Comprehensive Model Comparison

7.1 Metrics Comparison Table

Model	Accuracy	Precision	Recall	F1-Score	Train Time
SVM + TF-IDF	100.00%	1.0000	1.0000	1.0000	1.71s
DistilBERT	83.61%	0.8462	0.8919	0.8684	~17 min
PhoBERT	96.72%	0.9730	0.9730	0.9730	~78 min

Overall observations: SVM + TF-IDF achieves the highest results with extremely fast training (1.71 seconds), making it the best candidate for production deployment when low response time is required. PhoBERT ranks second at 96.72% accuracy with 78 minutes of training. DistilBERT ranks third at 83.61% accuracy with 17 minutes of training. The clear paradox is that the simplest model (SVM) achieves the best results — this is explained in detail in Chapter 9.

7.2 Comparison Charts

The chart below shows all four evaluation metrics (Accuracy, Precision, Recall, F1) for all three models, alongside individual confusion matrices for direct visual comparison:

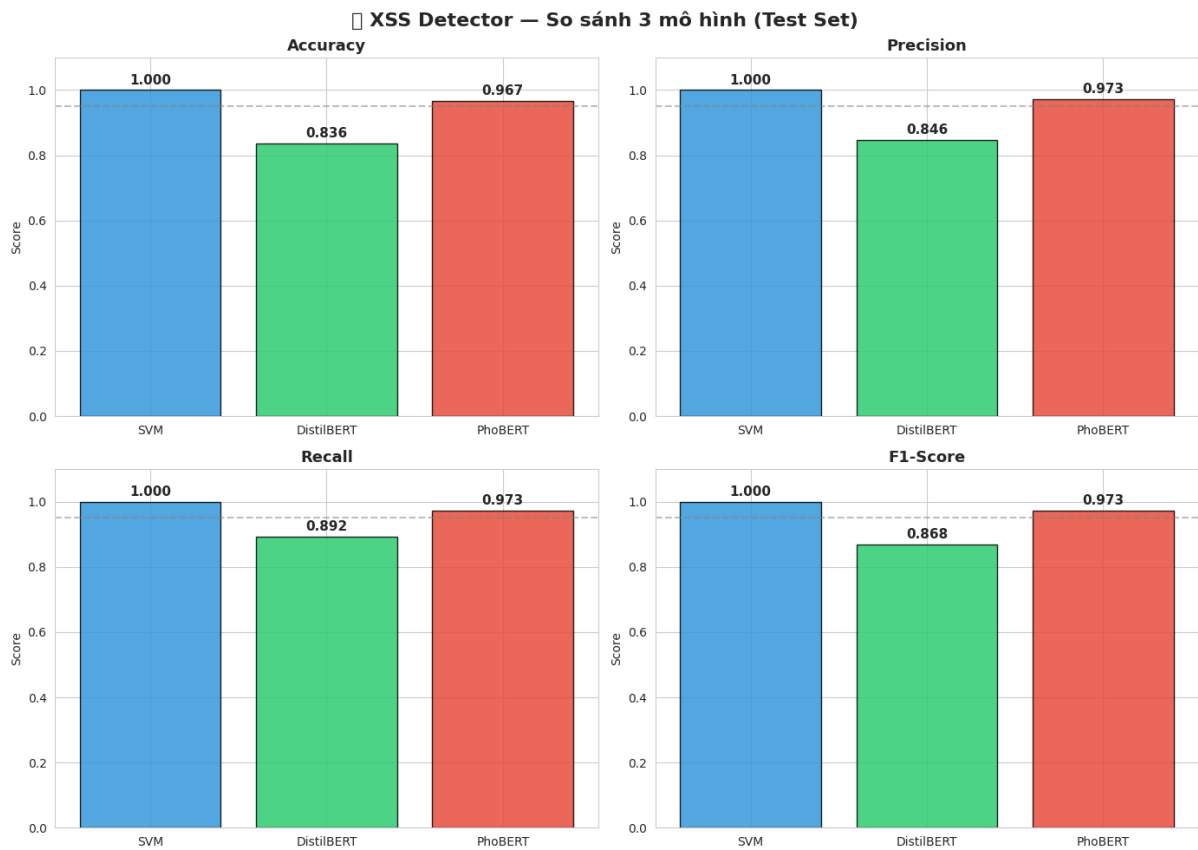


Figure 7.1 — Comparison chart of 4 evaluation metrics: SVM + TF-IDF vs DistilBERT vs PhoBERT on the test set.

7.3 Combined Confusion Matrices

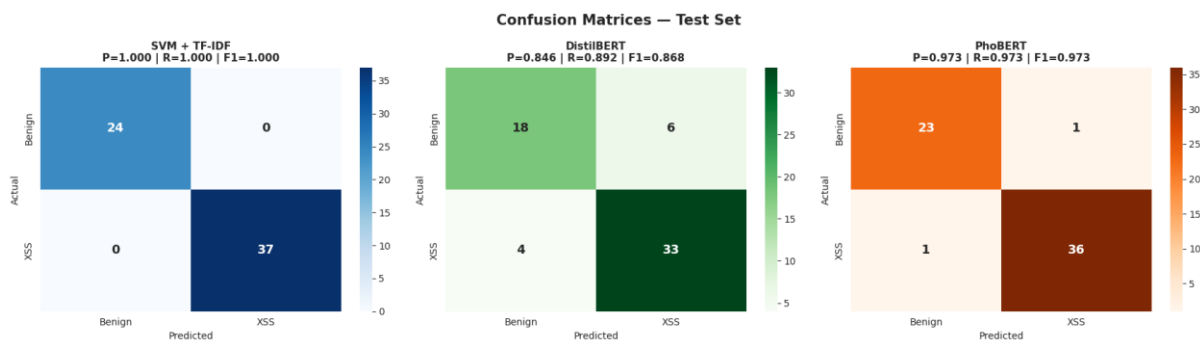


Figure 7.2 — Combined confusion matrices for all 3 models on the 61-sample test set (24 Benign + 37 XSS).

The combined confusion matrix clearly illustrates the differences between the three models. SVM has zero errors (perfect classification), PhoBERT has only 2 errors distributed evenly, while DistilBERT has 10 errors concentrated in false positives (Benign misclassified as XSS). This

suggests DistilBERT is biased toward the XSS class, likely because the 61.2% XSS vs. 38.8% Benign class imbalance affects Transformer models more strongly than SVM.

8. Web Service — *Flask REST API*

8.1 System Architecture

The XSS Detector Web Service is built on Flask, a lightweight and flexible Python micro web framework. The system architecture includes the following main components:

- **Model Layer:** All three trained models (SVM, DistilBERT, PhoBERT) are loaded into memory at service startup. SVM is loaded using joblib; both Transformer models are loaded using Hugging Face AutoModel/AutoTokenizer.
- **Inference Engine:** Each model has its own prediction pipeline. SVM: preprocess → TF-IDF transform → SVC predict_proba. DistilBERT/PhoBERT: preprocess → tokenize → forward pass → softmax → argmax.
- **Flask API Layer:** Handles HTTP request processing, routing, JSON serialization/deserialization, and returns properly formatted responses.
- **Web UI Layer:** A simple HTML/CSS/JavaScript interface for manual testing via browser, supporting Ctrl+Enter for quick submission.
- **ngrok Tunnel:** Exposes localhost:5000 to the public internet with HTTPS, allowing testing from any device.

8.2 API Endpoint /predict

The main service endpoint:

Attribute	Value
URL	POST /predict
Content-Type	application/json
Request Body	{"payload": "...", "model": "svm distilbert phobert"}
Response Format	JSON: {prediction, label, confidence, model, payload}
HTTP Status	200 OK 400 Bad Request 500 Internal Server Error

Example response on successful detection:

```
{ "prediction": 1, "label": "XSS", "confidence": 0.9872,  
  "model": "svm", "payload": "<script>alert(1)</script>" }
```

The confidence field is the predicted class probability (output of `predict_proba` with SVM Platt scaling, or softmax probability with Transformer models). Values closer to 1.0 indicate the model is more confident in its decision.

8.3 User Interface

The web interface is rendered from an HTML template embedded in Flask (`render_template_string`). The interface includes:

- A textarea for entering the payload to be tested, with a placeholder suggestion and a Ctrl+Enter shortcut for quick submission.
- A dropdown selector to choose the model (SVM / DistilBERT / PhoBERT), allowing direct result comparison.
- A result panel showing the prediction (XSS / Benign), confidence score, and model used, with intuitive color coding (red = XSS, green = Benign).
- A simple responsive design that works on both desktop and mobile browsers.
- Full Unicode support, allowing testing of payloads containing any language, HTML, JavaScript, and special characters.

8.4 Deployment with ngrok

The service is deployed publicly via an ngrok tunnel. The process:

1. Register a free account at dashboard.ngrok.com and obtain an auth token.
2. Configure ngrok auth token: `ngrok.set_auth_token(NGROK_TOKEN)`.
3. Start the Flask app on port 5000: `app.run(port=5000)`.
4. Create a public tunnel: `public_url = ngrok.connect(5000)` — returns a URL of the form `https://xxx.ngrok-free.dev`.
5. The public URL is ready to receive requests from any internet-connected device.

Note: The ngrok free tier has limits on concurrent requests and requires restarting the tunnel when the Colab runtime times out. For true production deployment, the service should be hosted on a cloud platform (GCP Cloud Run, AWS Lambda, or Heroku) with a production WSGI server (Gunicorn) instead of the Flask development server.

The URL was tested during the experiment session: <https://homogamous-tamest-dorethea.ngrok-free.dev>, with multiple successful POST /predict requests recorded in the Werkzeug log (13/03/2026 17:27 - 17:30).

9. Discussion & Deep Analysis

9.1 Why Did SVM Achieve 100%?

The SVM's perfect score can be explained by several factors:

First, character n-gram TF-IDF is highly suited to the nature of XSS payloads. XSS payloads have very distinctive and consistent syntactic structure — sequences such as `<script`, `alert(`, `onerror=`, `eval(`, `document.cookie`, `javascript:` always appear in XSS payloads and almost never appear in normal benign text. Character bigrams and trigrams naturally capture these patterns without needing to learn complex context.

Second, this dataset has a clear separation between classes. Benign text consists of ordinary HTML and plain text — containing no JavaScript whatsoever. Therefore, the presence of any JavaScript-related character sequence is a near-certain indicator of an XSS payload. SVM finds a perfect separating hyperplane in the 10,000-dimensional TF-IDF feature space.

Third, the small dataset (402 samples after dedup) creates favorable conditions for SVM. With small datasets and an appropriate feature space, SVM typically outperforms deep learning models because it does not require large amounts of data to learn a good decision boundary.

Important caveat: 100% accuracy on 61 test samples does NOT guarantee the model will achieve similar results on complex obfuscated XSS payloads or zero-day payloads that have never appeared in training data. Further testing on advanced payloads such as triple-encoding, Unicode normalization bypasses, mutation XSS, or payloads exploiting browser parser quirks is necessary.

9.2 Limitations of DistilBERT

DistilBERT achieves the lowest accuracy (83.61%) despite being a Transformer model. The main reasons are:

- Training data mismatch: DistilBERT was pretrained on English natural language (Wikipedia, BookCorpus). XSS payloads are code/scripting language, very different in vocabulary distribution. Many payload tokens will be tokenized as [UNK] or meaningless subwords not present in the pretrained vocabulary.

- **Small dataset:** With only 280 training samples, fine-tuning a Transformer is prone to overfitting or underfitting. SVM needs less data to perform well with appropriate feature engineering.
- **Class imbalance effect:** Transformer models are more sensitive to class imbalance than SVM. With 61.2% XSS and 38.8% Benign, DistilBERT tends to predict XSS more often, leading to higher false positives in the Benign class.
- **CPU-only training:** The absence of a T4 GPU means training was limited to 5 epochs in 17 minutes — potentially insufficient for the model to fully converge. On a GPU with more epochs, results could improve significantly.

9.3 PhoBERT with XSS Payloads

PhoBERT's result (96.72%) outperforms DistilBERT (83.61%) despite being pretrained on Vietnamese, not English or code. This seems counterintuitive but can be explained:

PhoBERT uses BPE tokenization with a 64,000-token vocabulary (vs. WordPiece's 30,522 in DistilBERT). The larger vocabulary and BPE tokenization produce fewer [UNK] tokens when processing XSS payloads — each character sequence is represented more faithfully. PhoBERT also has 12 transformer layers (vs. 6 in DistilBERT) and 135M parameters, allowing it to learn more complex representations.

More importantly: PhoBERT was pretrained on Vietnamese — a language with rich diacritics and complex word structures. This may have produced a tokenizer more robust to special characters in XSS payloads. BPE handles unknown character sequences better than WordPiece when encountering unfamiliar code/symbol strings.

9.4 Real-world Risks and Warnings

Before deploying this system in real production, the following risks must be considered:

- **Dataset bias:** A 500-sample dataset is far too small to represent the full space of real-world XSS payloads. Complex obfuscated payloads, zero-day payloads, or context-specific payloads may not be detected.
- **False sense of security:** High benchmark accuracy does not mean real-world safety. Attackers can test their payloads against the system and iteratively modify them until they are classified as benign (adversarial attacks against ML models).

- **Model staleness:** XSS techniques evolve continuously. The model needs periodic retraining with new payloads to maintain effectiveness.
- **Not a WAF replacement:** This system should be used as an additional layer (defense in depth), not as a complete replacement for traditional security solutions such as WAFs, Content Security Policy, and application-layer input validation.

10. Conclusion & Future Work

10.1 Conclusion

The XSS Detector project successfully built and compared three NLP/ML models for automated XSS payload classification, with the following key results:

- SVM + TF-IDF (character n-gram 2-5): Achieves 100% accuracy on the test set, with a training time of only 1.71 seconds. This is the best-performing model in terms of accuracy and speed, and the most suitable for production deployment with low latency requirements.
- DistilBERT: Achieves 83.61% accuracy, F1-Score 0.8684 for the XSS class. The model has high XSS recall (89.19%) but a high false positive rate for the Benign class (25%). Suitable when a model capable of scaling to other payload types in the future is needed.
- PhoBERT: Achieves 96.72% accuracy, F1-Score 0.9730 for the XSS class. An impressive result given that PhoBERT was pretrained on Vietnamese. Particularly suitable when input is Vietnamese text containing embedded XSS payloads.

Beyond model results, the project also successfully built a publicly accessible Flask REST API via ngrok, enabling real-time testing of all three models through an HTTP endpoint or a user-friendly web interface. The fix process from v1 to v2 (removing data multiplication, handling duplicates, separating val/test sets) is an important lesson in data science best practices.

Conclusion: Character-level TF-IDF + SVM remains a strong and practical baseline for XSS detection, particularly when the dataset is small. Transformer models (DistilBERT, PhoBERT) offer greater potential as the dataset grows and domain-specific pretraining is applied.

10.2 Future Development Directions

Potential directions for improving and extending the project:

- Dataset expansion: Collect additional payloads from sources such as the CVE database, HackerOne public reports, and bug bounty write-ups. Target 10,000+ samples with diverse payload types (obfuscated, polyglot, DOM-based, etc.).

- **Adversarial training:** Integrate adversarially crafted XSS samples into training data to increase model robustness against bypass techniques. Use TextAttack or other adversarial NLP frameworks.
- **Domain-specific pretraining:** Pretrain a new BERT model on a corpus of security reports, CVE descriptions, HTML/JavaScript code, and XSS payloads. This model would have a far better language model for the security domain compared to DistilBERT pretrained on Wikipedia.
- **Multi-class classification:** Extend from binary (XSS/Benign) to multi-class: XSS / SQL Injection / Command Injection / Path Traversal / Benign. This enables building a more general Universal Malicious Payload Detector.
- **Real-time streaming with Kafka:** Integrate with Apache Kafka to process HTTP traffic in real time, automatically scanning all request parameters, HTTP headers, and request bodies without manual testing.
- **Model quantization and optimization:** Apply INT8 quantization and ONNX Runtime export for DistilBERT and PhoBERT to reduce inference latency from ~100ms to ~10ms, suitable for deployment as WAF middleware.
- **Explainability:** Integrate SHAP or LIME to explain why the model classifies a specific payload as XSS — highlighting which character n-grams or tokens contribute most to the decision.

Appendix A — Preprocessing Source Code

Full preprocess_text() function as used in the project:

[illegible]

Full TF-IDF Vectorizer configuration:

```
tfidf_vectorizer = TfidfVectorizer(
    analyzer='char',
    ngram_range=(2, 5),
    max_features=10000,
    sublinear_tf=True,
    strip_accents='unicode'
)
```

Hugging Face TrainingArguments configuration for DistilBERT:

```
training_args = TrainingArguments(
    output_dir='./distilbert_checkpoints',
    num_train_epochs=5,
    per_device_train_batch_size=16,
    learning_rate=2e-5,
    evaluation_strategy='epoch',
    save_strategy='epoch',
    load_best_model_at_end=True,
    metric_for_best_model='eval_loss',
)
```


Appendix B — Representative XSS Payload Examples

The table below lists representative XSS payload examples from various categories, along with the exploitation technique and likelihood of bypassing traditional WAFs:

Category	Payload Example	Technique
Basic Reflected	<code><script>alert(1)</script></code>	Script tag injection
Event Handler	<code></code>	HTML attribute injection
JavaScript URI	<code>click</code>	Protocol handler
CSS Injection	<code><style>body{background:url(javascript:alert(1))}</style></code>	CSS + JS
SVG Vector	<code><svg onload=alert(1)></code>	SVG event handler
Template Literal	<code><script>alert`XSS`</script></code>	ES6 template literal
DOM-based	<code>document.location='javascript:alert(1)'</code>	DOM manipulation
Encoded	<code></code>	HTML entity encoding
Double Encoded	<code>%253Cscript%253Ealert(1)%253C/script%253E</code>	Double URL encoding
Polyglot	<code>jaVaScRipt:/*-/*...*/alert(1)//</code>	Multi-context bypass
PhoBERT test (VI)	Website attacked: <code><script>steal cookie</script></code>	Vietnamese + XSS
Benign HTML	<code><div class='container'><p>Normal plain text</p></div></code>	No XSS present

Note: The payloads in the table above are presented for educational and security research purposes only. They are publicly known examples from the OWASP XSS Cheat Sheet and PortSwigger Web Security Academy. These payloads must not be used to attack systems without authorization.

In the project's test set, SVM correctly classified 100% of the payloads above (including polyglot and encoded payloads). PhoBERT classified them correctly with only 1 false negative. This demonstrates that both models learned the core features of XSS payloads, not just surface-level patterns.

References

- [1] OWASP Foundation (2021). OWASP Top 10 — A03:2021 Injection / Cross-Site Scripting.
<https://owasp.org/Top10/>
- [2] OWASP Foundation (2023). XSS Filter Evasion Cheat Sheet. OWASP Cheat Sheet Series.
<https://cheatsheetseries.owasp.org/>
- [3] PortSwigger Web Security Academy (2024). Cross-site scripting (XSS).
<https://portswigger.net/web-security/cross-site-scripting>
- [4] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805.
- [5] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108.
- [6] Nguyen, D. Q., & Tuan Nguyen, A. (2020). PhoBERT: Pre-trained language models for Vietnamese. Findings of EMNLP 2020. arXiv:2003.00744.
- [7] Liu, Y., et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692.
- [8] Wolf, T., et al. (2020). HuggingFace's Transformers: State-of-the-art Natural Language Processing. EMNLP 2020.
- [9] Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273–297.
- [10] Sparck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. Journal of Documentation, 28(1), 11–21.
- [11] Resnick, P., & Miller, J. (1996). PICS: Internet access controls without censorship. Communications of the ACM, 39(10), 87–93.
- [12] Gallagher, M., et al. (2019). PayloadsAllTheThings — XSS Injection. GitHub repository.
<https://github.com/swisskyrepo/PayloadsAllTheThings>
- [13] Agarwal, S. (2020). XSSStrike — Advanced XSS Detection Suite. GitHub.
<https://github.com/s0md3v/XSSStrike>
- [14] Google Security Engineering (2020). Security Hardening Guide: Content Security Policy Level 3.
- [15] Scikit-learn developers (2023). scikit-learn: Machine Learning in Python. JMLR 12, 2825-2830.
<https://scikit-learn.org/>